Implementing the Complete Tip Flow for TipJar+

TipJar+ is a comprehensive Web3 tipping platform that integrates on-chain smart contracts, Circle's APIs (programmable wallets, gas sponsorship, paymaster, cross-chain transfers), a NestJS backend, and a Next.js frontend. The goal is to enable fans to tip content creators in USDC with minimal friction while the platform earns fees at various points. Below we break down the implementation of each component (smart contracts, Circle integration, backend, frontend, security) and describe the project structure and deliverables. Each section is informed by the provided strategy documents and project files.

1. Smart Contracts Implementation

We will develop three Ethereum smart contracts – TipProxy, WithdrawProxy, and DepositProxy – to handle on-chain tip transactions and fee collection. All contracts will be written in Solidity (with OpenZeppelin libraries for security) and thoroughly unit-tested. They include built-in fee logic (3.5% or 2.5% as specified), admin configuration of addresses, and deployment to a testnet (Polygon Mumbai or Ethereum Goerli for example). Security features (reentrancy guards, input validation, only authorized access) will be in place to protect user funds.

TipProxy (3.5% fee from Fan): This contract facilitates external crypto tips. When a fan with an external wallet (EOA) tips a creator, instead of sending USDC directly to the creator, the fan calls TipProxy.tip(creator, amount) (after approving the TipProxy to spend their USDC). The contract then transfers 96.5% of the amount to the creator's address and 3.5% to the platform's fee treasury address. This split ensures the platform fee (3.5%) is taken at the time of the tip from the fan. If the creator's preferred network differs from the fan's network, the TipProxy can coordinate with Circle's Cross-Chain Transfer Protocol (CCTP) – e.g. receiving USDC on Ethereum, then burning and minting via CCTP to deliver to the creator on Polygon. The TipProxy contract may emit an event for backend monitoring and includes safeguards like only accepting a supported stablecoin (USDC) and using SafeERC20 for transfers to handle token quirks.

WithdrawProxy (3.5% fee on Creator Withdrawals): This contract handles creators withdrawing their accumulated tips from the platform to an external address. If a creator's funds are held in a smart contract or escrow on-chain, the creator would invoke a withdraw function on this contract, specifying their desired destination (crypto wallet or bank via off-ramp). The WithdrawProxy then transfers 96.5% of the balance to the creator's target and 3.5% to the platform's fee address. This ensures the platform earns a fee when creators cash out. In practice with Circle, creators' funds are custodied in Circle wallets, so the actual withdrawal is initiated via Circle's Payouts API (see Section 2) rather than an on-chain call. However, for completeness and future extensibility, a WithdrawProxy contract is provided if on-chain custody is used. It includes permission checks so that only the legitimate fund owner (or the platform on behalf of the creator) can trigger a withdrawal.

DepositProxy (2.5% fee on Fiat On-Ramp): This contract receives USDC from fiat on-ramp providers and deducts a 2.5% fee. For example, if TipJar integrates with an on-ramp like Revolut or MoonPay that delivers USDC on-chain, the user can be given a deposit address associated with this contract. When USDC arrives, the DepositProxy forwards 97.5% to the

user's internal wallet (or a specific address tied to the user) and directs 2.5% to the platform fee address. This on-chain fee collection covers processing costs of fiat conversions. The contract might maintain a registry of pending deposits or use encoded data (memo or separate function call) to identify the user – or more simply, each user could have a unique instance of DepositProxy deployed. In our implementation, we prefer a single DepositProxy with a function depositFor(user, amount) that the backend or the on-ramp widget calls (if possible) to credit a specific user. If an on-ramp cannot call a function with parameters, an alternative is generating one-time deposit addresses or listening for Circle's incoming transfer webhooks to credit the appropriate user (as described in Path 4 of the flow).

Smart Contract Security and Configurability: All contracts include standard security patterns: ReentrancyGuard (to prevent reentrancy attacks on fee collection), input validations (e.g. non-zero addresses, positive amounts), and an owner/admin role for configuring platform addresses and fees. For example, the platform's treasury address (where fees accumulate) can be set by an admin, and the fee percentage might be adjustable (capped at some max). Unit tests will cover normal scenarios (correct fee deduction, transfers sum up correctly) and edge cases (zero amount reverts, only owner can change config, etc.). We will deploy these contracts to a testnet and verify them. The addresses of deployed contracts will be configurable in the backend and frontend so that transactions route correctly in different environments.

2. Circle API Integration

TipJar+ heavily leverages Circle's suite of services – Programmable Wallets, Gas Station, Paymaster, Transfers/Payouts API, CCTP, and Webhooks – to provide a seamless user experience. Below is how each is integrated:

Circle Programmable Wallets (Custodial DCW): When a creator signs up, the backend calls Circle's createWallet API to provision a custodial wallet for that creator. TipJar (the platform) is the technical controller of this wallet, while the creator is the beneficial owner. These wallets are Smart Contract Accounts (SCA) on EVM chains, which means they are actually smart contract wallets that Circle manages on behalf of the user. We also have the option to create similar wallets for fans who register (this was marked as a strategic decision – enabling fans to have internal wallets). Each Circle wallet comes with a unique blockchain address for USDC (e.g., mainWalletAddress in our database stores this). The Circle Wallet API and the @circle-fin/developer-controlled-wallets SDK are used to perform actions such as checking balances and initiating internal transfers.

Circle Gas Station (Gas Sponsorship): Circle's Gas Station service allows TipJar to pay for the gas fees of transactions originating from these custodial wallets. The platform maintains a deposit of MATIC/ETH in Circle's gas account; whenever a DCW (e.g., a creator's wallet) sends a transaction on Polygon or Ethereum, Circle deducts the gas cost from the platform's gas balance, so the user doesn't need native tokens. In our implementation, when the backend requests a transfer from a fan's Circle wallet to a creator's Circle wallet (internal tip), or a payout from a creator's wallet to an external address, no gas is required from the user – it's covered by Gas Station automatically. We will implement monitoring of the gas usage to manage costs and top-up the gas account when needed (including possibly an

internal alert if gas funds run low). The Gas Station charges a 5% fee on the gas amount used – this is a minor cost that the platform will factor into its 3.5% fees (negligible relative to tip amounts).

Circle Paymaster (ERC-4337 Gas Sponsorship for EOAs): For fans who prefer using their own crypto wallet (EOA) rather than a custodial wallet, we integrate Circle's Paymaster service (v0.8) to eliminate the need for the fan to hold MATIC/ETH for gas. This uses Account Abstraction (ERC-4337) behind the scenes. The frontend will construct a UserOperation object when an external fan chooses "Pay with USDC (External Wallet)" on the tip modal. This user operation targets the TipProxy contract's tip function, and specifies Circle's Paymaster as the paymaster. The frontend also uses the USDC permit mechanism to allow the Paymaster to pull the required USDC from the user's wallet. Essentially, the fan signs two things: (a) the UserOp (which includes the intended USDC transfer to the creator via TipProxy), and (b) an ERC-2612 Permit granting Circle's Paymaster permission to deduct the needed USDC for fees and the tip amount. The signed UserOperation is sent to an ERC-4337 bundler (e.g., a public bundler service). The bundler sends it to the EntryPoint contract on-chain, which calls the Circle Paymaster. The Paymaster pays the gas in MATIC (from Circle's gas account) and then uses the permit to charge the equivalent gas fee in USDC from the fan's wallet. Finally, the EntryPoint executes the user's intended action: calling TipProxy to transfer the USDC (minus 3.5%) to the creator. This way, the fan's MetaMask can send a tip with zero MATIC/ETH, spending a bit of extra USDC for gas (Circle adds ~10% overhead on gas fees for this service). Our backend will provide the frontend with the necessary config (addresses for EntryPoint, Paymaster, USDC), and will monitor the transaction status either via webhooks (if the creator's DCW gets the funds, Circle will webhook) or by polling the blockchain for the txHash. Integrating the Paymaster significantly improves UX for Web3-savvy fans by allowing gas fees in USDC. We will also implement fallbacks or alternate bundlers to avoid reliance on a single bundler service.

Circle Transfers & Payouts (Off-Ramp): When a creator wants to withdraw their earnings, the backend uses Circle's Transfers/Payouts API to move USDC from the creator's Circle wallet to an external destination. Two main off-ramp options are supported: crypto transfer (to the creator's external USDC address) and fiat payout (to the creator's bank account via ACH or wire). For on-chain crypto withdrawals, we call Circle's Transfer API specifying the source wallet (creator's DCW) and destination blockchain address (provided by the creator). Circle will then orchestrate an on-chain transfer from the SCA wallet to that address, using Gas Station to cover gas. For fiat payouts, Circle's Payouts API is used: the creator would have to input their bank details (and pass any required KYC), and the backend will initiate a payout which converts USDC to fiat and sends it to their bank. In both cases, we apply the WithdrawProxy's 3.5% fee logic: e.g., if a creator requests 100 USDC to an external wallet, the backend will actually initiate a transfer of 96.5 USDC to the creator's address, and separately move 3.5 USDC to the platform's treasury Circle wallet (this could be done via two Circle transfers: one to the user, one to platform). Alternatively, the backend could deduct the fee internally (reduce the amount available to withdraw). We record the payout in the database with status PENDING. Circle will send a webhook when the transfer is completed or if it fails, allowing us to update the status to CONFIRMED and inform the user in real-time. By leveraging Circle's payout rails, TipJar avoids managing private keys or liquidity for off-ramps – Circle handles the conversion and compliance.

Circle Cross-Chain Transfer Protocol (CCTP): TipJar+ supports cross-chain tipping: if a fan holds USDC on one chain and the creator prefers another chain, we seamlessly bridge the funds using Circle's CCTP. CCTP works by burning USDC on the source chain and minting on the destination chain, coordinated via Circle. In practice, our backend (or smart contract) will detect if cross-chain is needed when a tip is initiated. For example, a fan on Ethereum chooses to tip a creator whose main wallet is on Polygon. The TipProxy contract can accept the USDC on Ethereum and immediately call Circle's CCTP burn function for the amount (this could be done via an integrated CCTP contract call, or we instruct Circle's API to handle it if available). Circle will then emit an event/webhook when the burn is observed and mint the equivalent USDC on Polygon into the creator's wallet. The process takes a bit longer than same-chain transfer (needs block confirmations for the burn), but the frontend will simply show the tip as processing and then success. Both fan and creator get the benefit of transacting in their preferred networks transparently, with TipJar bridging behind the scenes. We will utilize Circle's developer documentation or SDK for CCTP to integrate this flow. If direct API support is limited, this might involve our backend invoking transactions on the burn/mint contracts using the custody wallet permissions.

Circle Webhooks: Webhooks are essential for keeping TipJar's backend in sync with asynchronous events from Circle. We will set up webhook endpoints (e.g. /api/webhook/circle) to receive events such as: wallet transfer completed, incoming payment, payout completed or failed, etc.. Specifically, when a DCW-to-DCW internal transfer is done (fan tips from internal balance), Circle will send a transfer.successful webhook with details (source wallet ID, destination wallet ID, amount). Our backend will verify the webhook's authenticity (using Circle's signing secret or by verifying the payload signature) and then mark the tip as completed in DB, trigger notifications, etc. For fiat on-ramp payments (if using Circle Payments API in future), webhooks like payment.successful or transfer.created would inform us that a card payment has been converted to USDC and deposited. For payouts, Circle's webhook will tell us if a payout to a bank is complete or if any compliance hold occurs. We implement idempotency on webhook handling – each event has an ID so we log and ignore duplicates to protect against replay. Webhooks will update the status of Tip and Payout records, and trigger real-time updates via WebSocket to the frontend (e.g. notify the creator that a new tip arrived or that their withdrawal is complete). In addition, for manual on-chain transfers (Path 4, where a fan independently sends USDC to a creator's public address), Circle's system will detect that incoming transfer into the custodial wallet and fire a webhook. Our backend will use that to credit the tip to the creator (since no platform fee was taken in that case, it's essentially a direct donation, but we still record it for the creator's stats).

In summary, Circle's services form the operational backbone of TipJar+. We handle wallet creation and internal transfers via the Wallets API, sponsor gas via Gas Station (for all DCW transactions) and via Paymaster (for external user transactions), allow fiat conversion in and out through Payments and Payouts, move funds across chains with CCTP, and rely on webhooks to maintain state consistency. Proper integration and testing with Circle's sandbox environment will be done to ensure all flows (tip, deposit, withdraw) work as expected even in various edge cases (network delays, webhook timeouts, etc.).

3. Backend (NestJS) Implementation

The backend is implemented in NestJS (Node.js/TypeScript) as a set of modules corresponding to TipJar's domains: tipping, payments, users, etc. It exposes a RESTful API for the frontend and handles business logic like validating transactions, calling Circle APIs, and persisting data. We will also incorporate a WebSocket or server-sent events mechanism for real-time notifications (using NestJS's WebSocket gateway for example) so creators see new tips instantly.

Key backend modules and endpoints include:

Tips Module: Handles tip transactions from fans to creators. This module exposes endpoints for the various tip methods:

POST /api/tips/internal – for logged-in fans using their internal Circle wallet balance to tip. This endpoint expects the tip amount and the creator's ID (or username) in the request, and the fan's auth token (JWT) in headers. It will:

1. Validate the JWT and get the fan's user record.

2. Check that the fan has a Circle wallet and sufficient USDC balance (by querying Circle or our DB's cached balance).

3. Create a transfer via Circle's Transfers API from the fan's Circle wallet to the creator's Circle wallet for the specified amount. (No fee is taken at this stage in an internal transfer, assuming platform fees will be taken on withdrawal or were taken during deposit; the internal tip is free aside from gas which is covered by Gas Station).

4. Mark the tip record in our database as INITIATED and respond to the frontend with a pending status.

5. When Circle's webhook confirms the transfer (or we poll for its completion), update the tip record to CONFIRMED and trigger a WebSocket event to the creator.

6. If any error occurs (insufficient balance, Circle API failure), return an appropriate error to the frontend (which will show the user a message).

POST /api/tips/external – for fans using an external wallet (EOA). This might not actually execute a tip directly, but rather provide the data needed to do so. In the purest form of ERC-4337 flow, the frontend can handle constructing and dispatching the user operation without backend involvement. However, our backend can assist by providing the current configuration:

It could return the required contract addresses, chain ID, and maybe a temporary nonce or session for security.

Alternatively, we might not need a dedicated endpoint if the frontend is pre-configured; instead we will have an endpoint to monitor or record an external tip after it's broadcast. For instance, POST /api/tips/external/confirm with the tx hash (or userOp hash) after the fan submits the tip. This allows our backend to start monitoring the blockchain for confirmation and also create a DB entry for the tip preemptively.

In any case, the backend will ultimately record the tip in the DB (with fromUserId = null if guest, method = USDC_ONCHAIN) and credit the creator once the on-chain transfer is detected either via our own blockchain listener or via Circle's webhook (since the creator's wallet is custodial, Circle can notify us of the incoming funds).

We also support POST /api/tips/fiat or include fiat tipping under a common endpoint. However, in our architecture, fiat on-ramp tipping is initiated on the frontend via an embedded widget or redirect (not by sending card details to our backend). So the backend's role is to handle the completion webhook. The fan on the website chooses e.g. "Pay with Card", completes the purchase through the on-ramp provider's flow (which knows the creator's address or wallet ID as the destination), and then we get a webhook (either from Circle if using Circle Payments API, or from the third-party like MoonPay) to confirm the payment. Thus, we may not need a direct REST endpoint for starting fiat payments (unless we use Circle's Payments API, where we'd create a payment intent via REST). In the MVP, if using a third-party on-ramp UI, no backend call is needed to start – only to finish (webhook).

Tip record creation and idempotency: When a tip is initiated (any method), the backend creates a Tip entry in the database with a unique ID, references to fan and creator, amount, payment method (enum: INTERNAL, EXTERNAL, FIAT, MANUAL), status INITIATED. It also stores any relevant transaction hash or Circle transfer ID. In the case of webhooks coming later, we ensure that processing the webhook matches an existing record or create one if needed (for manual tips, the webhook might arrive without a prior record, so we treat that as a new tip with fromUserId = null). To avoid double-counting, webhooks include transaction IDs which we can use to de-duplicate events (idempotent handling). Each tip record moves to CONFIRMED or FAILED state based on final outcome.

Withdrawals/Payouts Module: Handles creators withdrawing funds. Endpoints:

POST /api/payouts (or /api/withdraw) – for a creator to request a payout. This is an authenticated endpoint (creator's JWT required). The request includes the amount to withdraw and the target: either a crypto address or a fiat destination (bank account id).

The backend first verifies the creator's available balance (from Circle). We may maintain an internal ledger of the creator's "available balance" separate from Circle to account for fees, but since all funds sit in Circle wallets, it's simplest to consider the Circle wallet balance as the source.

It calculates the 3.5% fee. For example, if the creator requests 100 USDC to a crypto address, fee = 3.5 USDC. We then prepare two operations: (a) transfer 96.5 USDC to the external address via Circle's Transfers API, (b) transfer 3.5 USDC to the platform's master wallet (or we could simply deduct it and not transfer, if we keep fees in the creator's wallet – but better to move it to a central wallet for accounting).

Execute the transfer(s) via Circle Payouts API. For a bank payout, it's a single API call to send (Circle will automatically handle the conversion from USDC to fiat and deliver to the saved beneficiary bank).

Create a Payout record in the DB with status INITIATED, amount, fee, method (crypto or fiat), and link to the creator's user ID.

Respond to the API call with a success (or if Circle API gave immediate error, respond with that error).

Later, handle the webhook from Circle: update the Payout record to COMPLETED or FAILED. If completed, decrement the creator's balance in our DB (if we track it) and perhaps send a notification email or WebSocket event to the creator ("Your payout of 96.5 USDC is complete.").

GET /api/payouts – the creator can list their past withdrawal requests and statuses (from our DB).

We also ensure that a creator cannot withdraw more than they have, cannot spam multiple withdrawals that exceed balance, and possibly enforce minimum or maximum withdrawal limits for safety/compliance. These rules will be part of the validation.

On-Ramp Deposits (Fiat) Endpoint: If in Phase 2 we integrate Circle's Payments API for fiat on-ramp, we might have an endpoint like POST /api/deposit for fans:

This would accept an amount in fiat or USDC that the fan wants to purchase. The backend then calls Circle's Payments API to create a checkout or payment intent (including the fee or exchange rate). Circle might handle KYC in the widget or via hosted pages. For MVP we might bypass implementing this and use a third-party widget, but we structure the code anticipating this.

Alternatively, if using a provider like MoonPay, their SDK might directly handle it. In that case, our backend just stores any relevant info post-transaction (the webhook will inform us of a deposit).

In either case, when a fiat deposit completes, either Circle or the provider will deposit USDC to a specified address (likely the fan's Circle wallet address, or a DepositProxy contract address). We then credit the fan's internal balance accordingly.

User Module: Manages users (creators and fans), authentication and profile data. Key points:

Auth: We support email/password and OAuth (Google, Twitch) for creators/fans, as well as Sign-In with Ethereum (for Web3 users). NestJS will use Passport strategies for Google and Twitch OAuth; upon success we create or find a user, issue a JWT. For SIWE, we verify the signature and nonce then similarly issue a JWT. This module also handles creator onboarding: when a new creator account is made, call Circle to create their wallet (store the returned circleWalletId and blockchain address in our DB). Possibly, if we decide to also create wallets for fans upon registration (optional), we do similarly. We ensure to store necessary fields like circleWalletId, mainWalletAddress, username, profile info etc..

Profile & Settings: Endpoints to get or update profile info (bio, avatar URL, goal, etc) for creators. The avatar and banner images might be uploaded to S3 or similar; the backend will provide a signed upload URL or use a NestJS file upload module, then store the file links.

Dashboard Data: Endpoints for creators to fetch their dashboard stats – total earned, number of tips, recent tips list. This is aggregated from the Tip records in the DB. Also an endpoint for fans to fetch their balance and history if they use an internal wallet (the history combines deposits and tips they sent).

Webhooks: The backend will have a controller to handle Circle webhooks (and possibly separate controllers for other providers' webhooks). For Circle, we might use a dedicated module or include it under payments. This will contain logic to verify the signature (Circle provides a header and secret for verification) and then route the event to the appropriate handler (tip transfer completed, payout completed, incoming funds, etc.).

Database and Models: We use a relational database via Prisma ORM (as indicated by dependencies) for storing persistent data:

User table: stores user info (id, name, email, etc.), role (fan or creator), auth info, plus Circle wallet IDs and addresses if applicable. It will have fields for profile (displayName, bio, avatar URL, banner URL, goalAmount, etc.).

Tip table: stores each tip transaction – including an auto UUID, references to fromUser (nullable for guest) and toUser (creator), amount (decimal), message (the fan can attach a note to the tip), timestamp, method (USDC_INTERNAL, USDC_EXTERNAL, CARD, MANUAL), status, and txHash or external reference if available. This lets creators view their tip history and fans view the tips they've given.

Payout table: records withdrawals by creators – with id, userId, amount, fee, target (address or bank id), status, timestamp, and any external reference (like Circle payout ID or bank transaction ID).

Possibly Deposit table: if we handle on-ramp deposits separately, to track when a fan added funds (amount, method, status).

We may also have other tables for things like OAuth tokens (if storing refresh tokens for Google/Twitch – though we avoid storing sensitive tokens if possible), and an AuditLog for security events. But primary focus is on Tip and Payout which are core to tipflow.

All database operations will be done via Prisma in NestJS services, and important events (new tip, completed payout) will be emitted for further processing (e.g., to send notifications).

Real-time notifications: Using NestJS WebSocket Gateway or similar, we implement that when a tip is confirmed (via webhook or on-chain event), the backend emits a WebSocket event to the creator's client. The frontend (creator dashboard) subscribed to this socket will then show a live notification of the new tip (and possibly update their balance in real-time). Similarly, when a withdrawal completes, notify the creator so they know funds are out. This real-time aspect enhances the user experience (creators get instant feedback of support).

Testing & Deployment: We will write unit tests for critical services (e.g., tip service logic for fee calculation, webhook handler idempotency, etc.). Integration testing might be done against Circle's sandbox if time permits (maybe using Circle's testnet USDC on Polygon). The NestJS app will be dockerized for deployment, and environment variables will be used for all sensitive config (Circle API keys, DB credentials, JWT secret, etc.).

In summary, the NestJS backend orchestrates the tip flow by providing endpoints for each step, ensuring the rules (fees, limits, auth) are enforced, interacting with Circle's API for actual fund movements, and keeping our database in sync with those operations. It acts as the glue between the on-chain/Circle world and the front-end UI, encapsulating the business logic of TipJar+.

4. Frontend (Next.js) Implementation

The frontend is a Next.js application (React + TypeScript) that delivers both the public-facing pages and the user dashboards (for creators and fans). We will structure the pages and components in a logical way, and apply the design system provided (as per UI documents). The key pages and components are:

Creator Onboarding (/creator/setup): This page allows a new creator to sign up and configure their account. It will include:

Registration form or OAuth buttons (Google, Twitch) and a wallet connect option (for SIWE). We leverage next-auth or a custom approach for OAuth; upon success, we get redirected with a JWT cookie from backend. For SIWE, we use a library (like viem or ethers) to prompt MetaMask signature and send it to /auth/siwe backend.

After auth, if the user is new, we show a setup form: pick a username (for their public profile URL, e.g. "tipjar.com/@myname"), enter display name, bio, upload avatar & banner images, set an optional fundraising goal (amount & description), etc. There will be components for

image upload (integrating with an upload API or third-party storage) and text inputs for profile info.

On submission, the data is sent to PUT /api/creator/profile (for example) to update their profile in the database. We also possibly call POST /api/creator/wallet if the backend hasn't created a wallet yet (but likely the wallet is created at signup automatically).

The page will guide them through connecting their payout method as well – e.g., "Add a bank account or crypto address for withdrawals" which could link to another section or prompt later in the dashboard.

This page ensures the creator has everything set to start receiving tips: profile completed and wallet ready.

Fan Onboarding (/fan/setup): If we allow fans to create accounts (for internal balances or just to save their identity), we provide a similar signup page for fans:

They can register with email/password or OAuth or SIWE as well. If they use SIWE, it effectively links their EOA as their account identity.

If the product strategy is to have fans mostly operate without logging in (since they can just visit a creator's page and pay), this section might be optional. However, the milestone doc suggests a fan dashboard with an internal wallet, so if a fan does register, we might create a Circle wallet for them too and allow them to preload it.

On the fan setup page, after basic registration, if we are offering an internal wallet, we'd explain and let them opt-in (e.g. "Create a TipJar Wallet to pay easily without gas fees"). If they agree, backend creates a wallet and we show them their deposit address and current USDC balance (0 initially).

We'll also provide a way to add funds: a "Add funds via credit card" button which opens the on-ramp modal (see On-Ramp Modal below). Additionally, show a QR code or address they can send USDC to (their deposit address) to top up.

Essentially, /fan/setup leads into the Fan Wallet page.

Fan Wallet Page (/fan/wallet maybe or integrated into setup): Here a logged-in fan can:

See their current USDC balance in their TipJar internal wallet.

See a history of their transactions: any deposits they made and any tips they sent (with links to the creators).

Have controls to deposit or withdraw. (We might not allow withdrawing fan's money back to fiat because that complicates KYC – likely we encourage them to just use balance for tips, not act as a bank. But if needed, they could send to their own crypto wallet via a transfer).

This page uses components like BalanceDisplay, TransactionHistoryList etc.

It also should highlight that using the internal wallet means no gas fees for tipping (since we sponsor via Gas Station) – selling point for them to use it.

If the fan chooses to close account or not use it, they can always just not use this wallet.

Public Creator Profile (/[username]): This is a public page (no auth required to view) where fans come to actually tip the creator. For example, tipjar.plus/@alice. It includes:

Creator's banner image and avatar, display name, and bio.

If the creator set a fundraising goal, display a progress bar like "X out of Y USDC raised towards [GoalDescription]".

The Tip form as the central feature: an input for amount (with perhaps presets and a minimum), an optional message field to accompany the tip, and then payment method options. We'll have a "Tip with Crypto" (USDC) button and a "Tip with Card" button, for example.

If the visitor is logged in as a fan and has internal balance, we also show an option "Tip from my TipJar Wallet (Balance: X USDC)". This corresponds to the internal transfer path (Path 1) – if selected, we simply call the backend API to tip internally without any crypto signature needed, and then show success.

If the visitor chooses Crypto (external), we use web3 integration: upon clicking, if they have MetaMask (or WalletConnect etc.), we prompt them to connect (if not already) and then we call our logic to create the UserOperation for the Paymaster. This likely involves using the @circle-fin/paymaster-sdk or making a custom call: the frontend will assemble the userOp with: sender = user's EOA (but in practice, they might deploy a ephemeral smart wallet? However, Circle's EIP-4337 solution possibly allows EOA through their system, as discussed). We might use the viem library (as the docs suggested) to craft a userOp and sign it with the user's private key. Then we send it to a bundler (likely we'll integrate an API from Circle or a third-party bundler). We will need to give feedback to the user: e.g. show a loader "Transaction processing…" and then confirm when done. The profile page will listen for either the blockchain event or a webhook signal via our backend (perhaps simplest is polling our backend which monitors the tx). Once confirmed, show a success message "Thank you for the tip!".

If the visitor chooses Card (fiat), clicking that might open a modal with an embedded Circle widget or redirect to a hosted checkout. In MVP, it could open a third-party widget (MoonPay/Ramp). For instance, we might integrate Ramp Network's widget where we pre-fill it to buy USDC on the target chain to the creator's address. Or if using Circle, we might have built a simple card form modal (but handling PCI compliance is complex, so better to use hosted solutions). The UI will handle the states: open modal, upon completion (which might redirect back or just close on success), show success message. We instruct the user "After

payment, funds will be delivered to the creator." and perhaps hide the tip form until confirmation. Since the actual confirmation comes via webhook, the UI could poll the backend for a status or wait for a WebSocket event indicating the tip is complete. Then it shows "Success!".

The profile page may also show a feed of recent supporters (if the creator chooses to display that) or top tippers, but that's extra. At minimum, it should update the goal progress and tip count in real-time after a successful tip (we can fetch the new totals via an API or from the WebSocket event carrying tip info).

This page uses components like CreatorCard (for avatar, name, bio), GoalProgressBar, and the TipForm (which includes amount input and the payment buttons).

QR Code Generator: The platform can provide creators with a QR code that encodes their profile URL or a direct tip link. Likely on the creator dashboard (under "Tools" or profile page) there's a button "Generate QR Code". This can open a modal showing a QR code image that encodes https://tipjar.plus/@username. We can use a library like qrcode.react or QRCode.js to generate it on the fly. The creator can download this QR to share in streams or printed media. Implementation: a simple React component <QRCode value={profileUrl} /> embedded in a nice frame with instructions. No backend needed except to supply the base URL if not hardcoded.

UI Components: We will create reusable components to be used across the above pages:

Avatar – displays a user's avatar image in a circle, possibly with a fallback to initials. Used on profile and in any list of tips/supporters.

Bio – a component to display a user's bio text safely (we'll sanitize it to prevent XSS). Possibly supports basic markdown for links.

TipForm – encapsulates the tip input and payment options. It manages the state of amount and message, and has sub-components or logic branching for each payment method. E.g., if user selects internal, it just calls the API; if external, triggers metamask flow; if card, triggers on-ramp modal.

PayoutForm – used in the creator dashboard "Withdraw" page. It lets creator enter amount to withdraw (with a max = available balance), choose method (maybe a dropdown: Crypto Address or Bank). If Crypto, show an address input; if Bank, perhaps show their linked bank or a message to link one (Circle requires adding beneficiaries out-of-band, but we could integrate an interface for that). On submit, calls the backend and then displays status (could integrate with our WebSocket to update "processing -> done").

BalanceDisplay – shows an amount of USDC with proper formatting and maybe conversion to fiat for reference. Used in dashboards.

TransactionList – could display a list of tips or payouts with details (date, amount, from/to). Creator dashboard history, and fan wallet history, will use this.

QRCodeModal – as described, displays a QR code for the creator's link.

OnRampModal – this component will handle the card payment flow. For example, if using Ramp, it might embed an <iframe> with Ramp's widget URL (including parameters like destination address = creator's USDC address). If using Circle's own, perhaps we pop up a form (though likely we lean on a third-party UI for phase 1). We will ensure to cover success/failure events from the widget: many on-ramp providers allow callback functions or redirect URLs; our frontend will catch those and act accordingly (e.g., close modal and inform user to wait for confirmation).

Creator Dashboard Pages: After signup, creators access a dashboard (likely under /creator/dashboard or simply /dashboard if we detect role). This is an authenticated SPA section where creators manage their account. The main sub-pages:

Dashboard Home: Overview with total earned USDC, number of tips, and a list of recent tips. Also important alerts (e.g., "Verify your email" or "Complete KYC for higher withdrawal limits") as needed.

Tips History: A page with a table of all tips received, with filters by date/amount, export CSV option.

Withdrawals: A page containing the PayoutForm to request withdrawal, and a list of past withdrawals with statuses.

Edit Profile: Page to update display name, bio, avatar, banner, social links, etc.. Possibly includes setting a custom username if not set.

Account Settings: Manage account-level settings like email, password, 2FA, connected accounts (Google/Twitch). Also where they can see their connected wallet (for SIWE users) or even switch to a different wallet.

Creator Tools: Page where they can get the embeddable widget code (an HTML snippet to put a mini tip button on their personal site), see their public profile link and QR code generator, and manage any integration like a Twitch bot or alerts (if planned).

The dashboard is implemented using a combination of Next.js pages and client-side components. We might use Next.js API routes for some authenticated calls, or just call the NestJS API directly from the client. Given that we already have a separate NestJS API, the Next app will mostly be a client to that API, rather than using Next's built-in API routes extensively.

We ensure that navigating within the dashboard doesn't do full page reloads unnecessarily – could use Next's dynamic routing or a single-page app approach with React state management (since after login the rest can be mostly client-side rendered). For simplicity, Next's default page routing is fine, possibly with some use of SWR or React Query to fetch data.

Fan Experience (without login): The majority of fans might not log in. They will simply go to a creator's profile and use one of the tip methods. We optimize that path to be as few clicks as possible. E.g., if clicking "Pay with MetaMask", it directly triggers the MetaMask flow. If "Pay with Card", directly opens the payment modal. After a successful tip, we show a nice thank-you and perhaps offer "Create an account to track your tips" or "Share this on Twitter". But we do not force account creation for one-time tippers. This lowers friction.

Styling and Layout: We will use the provided design (the "Design System (UI/UX)" referenced in docs). Likely a modern, clean design with a consistent color palette (perhaps the palette.txt is the design tokens for colors). The Next.js app will use a global CSS (or CSS-in-JS with styled-components or Chakra UI depending on preference). We aim for responsive design so that the public profile and basic tipping works on mobile devices too (important if fans scan a QR code on their phone). We'll implement the necessary UX touches like loading spinners on actions, error messages (e.g., if MetaMask tx is rejected or if card payment fails), and input validation on forms.

SEO and SSR: The creator profile pages should be server-side rendered for SEO benefits (so search engines can index creators' pages). Next.js will handle this by default for the [username] dynamic route – we fetch the creator's profile data at build or request time and render the page with their name and bio in meta tags, etc. The landing page (if any) is also SSR. The dashboard pages can be mostly client-side (since they require auth, SEO is not relevant).

Frontend Integration with Backend: We configure environment variables for the API base URL, and for any keys needed (perhaps the on-ramp widget API key if needed on frontend). We ensure CORS is set so the Next app (running on say domain front.tipjar) can talk to the API (api.tipjar). For Web3 interactions, we include the necessary libraries (ethers or viem for signing, and possibly a library from Circle for Paymaster if available). We also import the ABI of TipProxy if needed to encode function calls (but the userOp might not require us to directly use the contract ABI if we send via bundler). Still, to get the correct function selector for tip(address,uint256), we might use ethers.js to encode the call that goes in the UserOp.

Throughout the frontend, we prioritize ease of use and clarity. For example, when a user chooses a method, we display informative messages: if using external crypto, we tell them "You will be prompted to sign a transaction and pay a small fee in USDC for network costs"; if using card, "Powered by [Provider], you may need to complete KYC for larger amounts". We also show success feedback including a link to a block explorer for on-chain transactions – e.g., "View on Polygonscan" – to leverage blockchain's transparency as a trust signal.

In summary, the Next.js frontend provides all necessary interfaces for fans and creators, integrates with web3 wallets for crypto payments, and presents a polished UI for the tip flow. It implements the core pages /creator/setup, /fan/setup, public profile /@username, and the various dashboard pages, using a set of reusable components for consistent design and functionality.

5. Security Considerations

Security is paramount across all parts of TipJar+. We implement multiple layers of protection and validation:

Input Validation & Amount Limits: All API inputs (REST body or query params) are validated using NestJS pipes or DTOs (ensuring types and ranges). Tip amounts and deposit amounts are capped to reasonable limits per transaction (e.g., perhaps max $1000 per tip by default) to mitigate mistakes or fraud. Similarly, we enforce minimum amounts (a few cents in USDC) to avoid dust transactions. On the contract level, the TipProxy and others check that the amount is > 0 and that addresses provided are not zero-address. The backend also cross-verifies that a fan isn't tipping more than their available balance (for internal tips). If a user tries to manipulate the frontend to send a larger amount, the backend will reject it if it exceeds their balance or set limits.

Authentication & Authorization: Only authorized users can hit certain endpoints. JWT auth (with HttpOnly cookies or Bearer tokens) is required for endpoints like tipping internally, withdrawing, viewing one's dashboard, etc. We implement role-based checks – e.g., only a creator can call the withdraw endpoint for their own account, and the server ensures the JWT's userId matches the target. For any admin-only operations (not much mentioned, but e.g., adjusting fees or viewing all data), we'd have an admin role. We also protect against ID tampering: even if a user is authenticated, they cannot withdraw funds on behalf of another or tip from another's wallet – our backend uses the JWT identity and internal mappings (like circleWalletId) rather than accepting those as client input.

CORS and CSRF: We configure CORS on the backend to allow only our frontend's origin (and perhaps Circle's domains for webhooks) to call the APIs. This prevents malicious sites from invoking our endpoints with user credentials. For CSRF, since our login might set a JWT cookie (if using cookie-based auth for OAuth flows), we will include CSRF tokens in state or use SameSite cookies to mitigate cross-site requests. Alternatively, if we use purely Bearer tokens stored in memory, CSRF is not an issue. Regardless, we ensure all state-changing requests require a proper auth token that a random third-party site cannot steal.

XSS Protection: The frontend will sanitize any user-generated content before rendering. For example, when displaying a user's bio or a fan's tip message on the site, we strip out or escape HTML tags to prevent script injection. We also sanitize data that goes into QR codes or widget code generation. Additionally, HTTP headers like Content-Security-Policy might be used to restrict script sources, and we avoid dangerously setting HTML in React.

Smart Contract Security: The smart contracts are simple but we still adhere to best practices. The fee calculations use safe math (solidity 0.8 has built-in overflow checks). We consider precision – since 3.5% of an amount might not be an integer if using smallest unit (cents), we clarify if "3.5%" means we operate in base currency with decimals. Likely USDC has 6 decimals, so 3.5% of 100 USDC = 3.5 USDC exactly; we implement fee = amount * 35 / 1000 to get a fractional fee. The remainder goes to creator. Any remainder due to division (if not perfectly divisible) can be kept by platform or given to creator – but since 0.5% steps of 100, it should be exact with one decimal in most cases. We also ensure the contracts use

ReentrancyGuard where needed (particularly DepositProxy which receives funds – though it immediately forwards, we guard reentrancy on that forward function). Only the contract owner (the platform's deployer) can set critical parameters (like changing the fee percentage or fee recipient). The contracts will be deployed on testnet and their addresses stored in backend config – no one can arbitrarily swap them since backend and frontend use known addresses.

Webhooks Security: We verify all incoming webhooks. Circle's webhooks come with a signature header and we have a shared secret (or we use their public key). We will compute an HMAC of the payload and compare, as per Circle's docs. This prevents attackers from forging webhooks. Additionally, each webhook event (Circle includes an id and type) is logged; we ignore duplicates by checking if we've processed that id before (to protect against replay). In case of a replay attack with a different id (unlikely if signature doesn't match), we still check if the content (transaction reference) was already handled in our DB and avoid double-spending or double-counting. For user-triggered actions like withdraw or internal tip, even though they are initiated by our system, we wait for webhook confirmation to finalize – which adds security since we trust Circle's confirmation rather than assuming success.

Prevention of Replay in Blockchain Transactions: The smart contracts use nonces or rely on globally unique transactions. For example, the Paymaster flow uses the standard ERC-4337 mechanism which inherently prevents replay by userOp hashes and the EntryPoint's handling of nonces for contract wallets. Also, since EOA uses permit, the permit is one-time with a nonce in the user's USDC allowance, so it cannot be reused maliciously. On our backend, if we generate any one-time addresses or links (like magic links or invite codes), we ensure they expire or are single-use.

Monitoring and Alerts: As a security measure, we will monitor unusual activities. For example, if a single fan tips an excessively large amount or many rapid tips, we might flag it for review (could be fraud or error). We also log all critical operations (transfers, payouts) with enough info to trace issues. In case of any failure in a Circle API call or a mismatched amount, the system should alert admins.

Dependency and Platform Security: We keep our packages (NestJS, Next.js, Circle SDK, etc.) updated to patch any known vulnerabilities. We serve the frontend over HTTPS always to protect data in transit. If we use a service like NextAuth, we carefully configure session cookies (HTTP-only, secure, SameSite). On the blockchain side, we are using well-known contracts (USDC token contract, Circle's EntryPoint/Paymaster) – we will verify their addresses from official sources.

In essence, we strive to make the system robust against both malicious attacks and inadvertent errors. The combination of secure contract code, rigorous backend validations, and safe frontend practices will ensure that the tip flow is trustworthy for all participants. TipJar's positioning as handling real money (USDC) means we cannot take shortcuts on security.

6. Project Structure and Deliverables

The implementation will be organized into a clear folder structure, separating different concerns. Each major component (contracts, backend, frontend, etc.) is packaged with its own README file explaining usage, configuration, and testing instructions. The proposed repository structure is as follows:

```
TipJarPlus/
├── contracts/
│   ├── contracts/            # Solidity source files
│   │   ├── TipProxy.sol
│   │   ├── WithdrawProxy.sol
│   │   └── DepositProxy.sol
│   ├── test/                 # Unit tests for smart contracts (JavaScript/TypeScript or Solidity)
│   │   ├── tipproxy.test.ts
│   │   ├── withdrawproxy.test.ts
│   │   └── depositproxy.test.ts
│   ├── hardhat.config.ts     # Hardhat or Foundry config (assuming Hardhat)
│   └── README.md             # How to compile, deploy, and test contracts
├── backend/
│   ├── src/
│   │   ├── main.ts           # NestJS bootstrap
│   │   ├── app.module.ts
│   │   ├── tips/             # Tips module
│   │   │   ├── tips.controller.ts
│   │   │   ├── tips.service.ts
│   │   │   └── tips.module.ts
│   │   ├── payouts/          # Payouts (withdrawals) module
│   │   │   ├── payouts.controller.ts
│   │   │   ├── payouts.service.ts
│   │   │   └── payouts.module.ts
│   │   ├── users/            # Users (auth, profiles) module
│   │   ├── webhooks/         # Webhooks handling module
│   │   ├── circle/           # Circle API integration services (could also be within other
modules)
│   │   ├── common/           # common utilities, guards, interceptors
│   │   └── ... (other modules like auth, etc.)
│   ├── test/                 # e2e tests or unit tests for services
│   ├── prisma/               # Prisma schema and migrations
│   │   └── schema.prisma
│   ├── nest-cli.json, tsconfig.json, etc.
│   └── README.md             # Setup (env vars for Circle keys, DB), run dev, run tests
├── frontend/
│   ├── pages/
│   │   ├── index.tsx         # Landing page or redirect to login
│   │   ├── [username].tsx    # Public creator profile page
│   │   ├── creator/
│   │   │   ├── setup.tsx     # Creator setup/onboarding
│   │   │   ├── dashboard.tsx # Could also be a folder if multiple subpages
```

```
│   │   │   └── ... (other creator pages)
│   │   ├── fan/
│   │   │   ├── setup.tsx       # Fan setup page
│   │   │   ├── wallet.tsx      # Fan wallet page
│   │   │   └── ...
│   │   ├── api/                # Next.js API routes (if any, e.g. for image proxy or auth callback)
│   │   └── _app.tsx, _document.tsx (Next.js globals)
│   ├── components/
│   │   ├── Avatar.tsx
│   │   ├── TipForm.tsx
│   │   ├── PayoutForm.tsx
│   │   ├── QRCodeModal.tsx
│   │   ├── BalanceDisplay.tsx
│   │   ├── etc...
│   ├── styles/                 # CSS or Sass files, or styled-components theme
│   ├── public/                 # static assets (images, icons)
│   └── README.md               # Instructions to run the frontend, env config (API base URL,
etc.)
├── circle/                     # (Optional) Scripts or config for Circle services
│   ├── seed-gas-account.ts     # script to fund gas station account
│   ├── verify-webhook.ts       # tool to test webhook signature verification
│   └── README.md               # Info on Circle integration, how to configure webhooks, etc.
├── utils/                      # Shared utilities or config across packages
│   ├── constants.ts            # e.g., fee constants, chain IDs
│   ├── helpers.ts
│   └── README.md               # Description of util functions
└── package.json (or pnpm/yarn workspaces config if monorepo)
```

Each folder has its own README:

contracts/README.md will describe each contract's purpose, how to compile (e.g., "Run npx hardhat compile"), how to run tests (npx hardhat test or using Foundry), and how to deploy to testnet (npx hardhat run scripts/deploy.ts --network mumbai). It will also mention any contract addresses or links to verification on block explorer for test deployments.

backend/README.md will cover setting up the NestJS server. This includes installing dependencies (npm install), setting environment variables (like DATABASE_URL for Postgres, CIRCLE_API_KEY, CIRCLE_WEBHOOK_SECRET, etc.), running migrations (npx prisma migrate deploy), and then starting the server (npm run start:dev). It also describes how to run the test suite (npm run test). Additionally, it can mention how to simulate webhooks (perhaps with a sample payload) and any known gotchas.

frontend/README.md will explain how to start the Next.js dev server (npm run dev), where to configure the backend API URL (maybe in next.config.js or .env.local), and how to build for production (npm run build && npm start). It will also list any required environment variables (e.g., for third-party widgets or analytics). Testing instructions (if we add Jest tests for components) can also be included.

circle/README.md will summarize how we've integrated Circle – e.g., "We use Circle's sandbox – set CIRCLE_API_KEY and ensure to whitelist IPs, configure webhooks at URL /api/webhook/circle in the Circle dashboard. Use circle/verify-webhook.ts to test the signature verification logic." It might also detail how to fund the Gas Station (manually via Circle's dashboard or API).

utils/README.md will list any shared utilities or how configuration is managed (for instance, if we have a config file that both backend and frontend import for certain constants like the fee percentages or supported chains).

Finally, we ensure that the entire output is ready to be deployed or handed off. The code will be properly formatted and linted. By structuring in distinct modules (creator vs fan vs common), future developers can easily navigate the project. Delivering in a monorepo style (if using npm workspaces) or just a structured repository allows running each part (contracts, backend, frontend) independently for testing, while also enabling integration tests (e.g., running the backend and hitting it with simulated requests, etc.).

Deployment considerations: We will include notes in the README about deploying to testnet (for contracts) and to cloud (for backend/frontend). For example: deploying the backend to Heroku or AWS, and that environment variables for Circle must be kept secret. Also, the frontend might be deployed to Vercel (since it's Next.js) – ensure to set env vars there.

By following the above plan, the TipJar+ tip flow implementation will be complete and robust, covering smart contracts, Circle integration, backend logic, frontend UI/UX, and security. Each component is delivered with documentation and can be tested in isolation and as a whole – ready to be integrated into the TipJar+ repository and taken for a test run by the team.